



(CSE 4802: Compiler Design Lab)

LAB 2

Report on Lexical Analyzer using LEX/Flex

Submitted by:

Md. Mosharrifur Rahman Ratul

Student ID: 210041213

Section: 2A

BSc in CSE, Dept of Computer Science and Engineering
Islamic University of Technology (IUT)

May 15, 2026

Contents

1	Introduction	2
2	Approach	2
3	Code Implementation	2
4	Test Input and Output Files	6

1 Introduction

A lexical analyzer is the first phase of a compiler. It reads the source program character by character and converts it into meaningful tokens such as keywords, identifiers, operators, constants, and delimiters. The output generated by the lexical analyzer is later used by the parser phase of the compiler.

LEX / Flex

LEX (or Flex) is a tool used for generating lexical analyzers automatically. A `.l` file is written using regular expression rules and actions. After processing the `.l` file using Flex, a C source file named `lex.yy.c` is generated. This generated file is then compiled to create the final executable lexical analyzer.

2 Approach

In this task, a lexical analyzer is implemented using Flex. Different token patterns such as keywords, identifiers, constants, operators, and delimiters are defined using regular expressions inside the `lexer.l` file.

The analyzer reads the input program from `input.txt`. When a pattern matches, the corresponding token type and lexeme are printed in the output stream. Invalid tokens and malformed constants are also detected and reported as lexical errors.

After scanning the complete input file, the analyzer generates a summary report containing the total number of tokens, operators, constants, identifiers, delimiters, and errors.

3 Code Implementation

The following `lexer.l` file is used for implementing the lexical analyzer using Flex:

Listing 1: `lexer.l` File

```
1 %{\n2 #include <stdio.h>\n3 #include <string.h>\n4
```

```

5 static FILE *out = NULL;
6
7 static int total_tokens = 0;
8 static int error_count = 0;
9
10 static int keyword_count = 0;
11 static int identifier_count = 0;
12
13 static int int_const_count = 0;
14 static int float_const_count = 0;
15 static int char_const_count = 0;
16 static int string_const_count = 0;
17
18 static int arith_op_count = 0;
19 static int rel_op_count = 0;
20 static int logical_op_count = 0;
21 static int bitwise_op_count = 0;
22 static int assign_op_count = 0;
23 static int incdec_op_count = 0;
24
25 static int delimiter_count = 0;
26
27 static void emit_token(const char *type, const char *lexeme) {
28     if (out) {
29         fprintf(out, "< %s , %s>\n", type, lexeme);
30     }
31     fprintf(stdout, "< %s , %s>\n", type, lexeme);
32     total_tokens++;
33 }
34
35 static void emit_error(const char *message, const char *lexeme)
36 {
37     if (out) {
38         fprintf(out, "[ERROR] Line %d: %s -> %s\n", yylineno,
39             message, lexeme);
40     }
41     fprintf(stdout, "[ERROR] Line %d: %s -> %s\n", yylineno,
42         message, lexeme);
43     error_count++;
44 }
45 %}
46
47 %option noyywrap
48 %option yylineno
49
50 KEYWORD      int|float|if|else|print|for|while
51 ID           [A-Za-z_][A-Za-z0-9_]*
52 INTCONST     [0-9]+
53 FLOATCONST   [0-9]+\.[0-9]+
54 CHARCONST    \'([^\n\']|\\.|\\.)\'

```

```

52 INVALID_CHARCONST \' ([^\\'\n]|\|.){2,}\'
53 STRINGCONST \" ([^\\\"\\n]|\|.)*\"
54 UNTERM_STRING \" ([^\\\"\\n]|\|.)*\n
55 UNTERM_CHAR \' ([^\\'\n]|\|.)*\n
56
57 %%
58
59 \"/\".* { }
60 \"/\"\\*( [^*] | \\*+ [^*/] ) * \\*+ \"/\" { }
61
62 [ \\t\\r\\n]+ { }
63
64 \"+\"|\"--\" { emit_token(\"INCDEC_OP\", yytext
        ); incdec_op_count++; }
65
66 \"<=\"|\">=\"|\"+=\"|\"-=\"|\"*=\"|\"/=\"|\"%=\"|\"&=\"|\"|=\"|\"^=\"|\"=\"
67 { emit_token(\"ASSIGN_OP\", yytext); assign_op_count++; }
68
69 \"==\"|\"!=\"|\"<=\"|\">=\"|\"<\"|\">\"
70 { emit_token(\"REL_OP\", yytext); rel_op_count++; }
71
72 \"&&\"|\"||\"|\"!\"
73 { emit_token(\"LOGICAL_OP\", yytext); logical_op_count++; }
74
75 \"<<\"|\">>\"|\"&\"|\"|\"|\"^\"|\"~\"
76 { emit_token(\"BITWISE_OP\", yytext); bitwise_op_count++; }
77
78 \"+\"|\"-\"|\"*\"|\"/\"|\"%\"
79 { emit_token(\"ARITH_OP\", yytext); arith_op_count++; }
80
81 \";\"|\", \"|\"(\"|\")\"|\"{\"|\"}\"
82 { emit_token(\"DELIMITER\", yytext); delimiter_count++; }
83
84 {KEYWORD}
85 { emit_token(\"KEYWORD\", yytext); keyword_count++; }
86
87 {FLOATCONST}
88 { emit_token(\"FLOAT_CONST\", yytext); float_const_count++; }
89
90 {INTCONST}
91 { emit_token(\"INT_CONST\", yytext); int_const_count++; }
92
93 {INVALID_CHARCONST}
94 { emit_error(\"Invalid character constant\", yytext); }
95
96 {UNTERM_CHAR}
97 { emit_error(\"Unterminated character constant\", yytext); }
98
99 {UNTERM_STRING}
100 { emit_error(\"Unterminated string constant\", yytext); }

```

```

101 {CHARCONST}
102 { emit_token("CHAR_CONST", yytext); char_const_count++; }
103
104 {STRINGCONST}
105 { emit_token("STRING_CONST", yytext); string_const_count++; }
106
107 {ID}
108 { emit_token("IDENTIFIER", yytext); identifier_count++; }
109
110 .
111 { emit_error("Unrecognized token", yytext); }
112
113 %%
114
115
116
117 int main(void) {
118     FILE *in = fopen("input.txt", "r");
119     if (!in) {
120         fprintf(stderr, "Failed to open input.txt\n");
121         return 1;
122     }
123
124     out = fopen("output.txt", "w");
125     if (!out) {
126         fprintf(stderr, "Failed to open output.txt\n");
127         fclose(in);
128         return 1;
129     }
130
131     yyin = in;
132
133     fprintf(out, "Token Stream:\n");
134     fprintf(out, "-----\n");
135     yylex();
136
137     int constants_count = int_const_count + float_const_count +
138         char_const_count + string_const_count;
139     int operators_count = arith_op_count + rel_op_count +
140         logical_op_count + bitwise_op_count + assign_op_count +
141         incdec_op_count;
142
143     fclose(out);
144     fclose(in);
145     return 0;
146 }

```

4 Test Input and Output Files

The following sample input file is used to test different types of tokens such as keywords, identifiers, constants, operators, delimiters, and lexical errors.

Listing 2: input.txt File

```
1 int main() {
2     int a = 10;
3     int b = 20;
4
5     float avg = 15.5;
6     char ch = 'A';
7
8     if(a < b && b != 0) {
9         avg += a + b;
10        a++;
11    }
12
13    print("Lexical Analyzer");
14
15    value = a << 2;
16    flag = a & b;
17
18    @invalid
19 }
```

For the above input file , the lexical analyzer gives the following output.txt file showing all the token streams, the analysis part showing the count or frequency of each of the token, token types etc. and also the error count and error details. The output.txt file is:-

Listing 3: output.txt File

```
1
2 Token Stream:
3 -----
4 <KEYWORD , int>
5 <IDENTIFIER , main>
6 <DELIMITER , (>
7 <DELIMITER , )>
8 <DELIMITER , {>
9 <KEYWORD , int>
10 <IDENTIFIER , a>
11 <ASSIGN_OP , =>
12 <INT_CONST , 10>
```

```

13 <DELIMITER , ;>
14 <KEYWORD , int>
15 <IDENTIFIER , b>
16 <ASSIGN_OP , =>
17 <INT_CONST , 20>
18 <DELIMITER , ;>
19 <KEYWORD , float>
20 <IDENTIFIER , avg>
21 <ASSIGN_OP , =>
22 <FLOAT_CONST , 15.5>
23 <DELIMITER , ;>
24 <IDENTIFIER , char>
25 <IDENTIFIER , ch>
26 <ASSIGN_OP , =>
27 <CHAR_CONST , 'A'>
28 <DELIMITER , ;>
29 <KEYWORD , if>
30 <DELIMITER , (>
31 <IDENTIFIER , a>
32 <REL_OP , <>
33 <IDENTIFIER , b>
34 <LOGICAL_OP , &&>
35 <IDENTIFIER , b>
36 <REL_OP , !=>
37 <INT_CONST , 0>
38 <DELIMITER , )>
39 <DELIMITER , {>
40 <IDENTIFIER , avg>
41 <ASSIGN_OP , +=>
42 <IDENTIFIER , a>
43 <ARITH_OP , +>
44 <IDENTIFIER , b>
45 <DELIMITER , ;>
46 <IDENTIFIER , a>
47 <INCDEC_OP , ++>
48 <DELIMITER , ;>
49 <DELIMITER , }>
50 <KEYWORD , print>
51 <DELIMITER , (>
52 <STRING_CONST , "Lexical Analyzer">
53 <DELIMITER , )>
54 <DELIMITER , ;>
55 <IDENTIFIER , value>
56 <ASSIGN_OP , =>
57 <IDENTIFIER , a>
58 <BITWISE_OP , <<>
59 <INT_CONST , 2>
60 <DELIMITER , ;>
61 <IDENTIFIER , flag>
62 <ASSIGN_OP , =>

```



```

63 <IDENTIFIER , a>
64 <BITWISE_OP , &>
65 <IDENTIFIER , b>
66 <DELIMITER , ;>
67 [ERROR] Line 18: Unrecognized token -> @
68 <IDENTIFIER , invalid>
69 <DELIMITER , }>
70
71
72
73
74 Summary Section:
75 -----
76 Total Tokens: 65
77 Keywords: 6
78 Identifiers: 19
79 Constants: 7
80     Integer Constants: 4
81     Float Constants: 1
82     Character Constants: 1
83     String Constants: 1
84 Operators: 14
85     Arithmetic Operators: 1
86     Relational Operators: 2
87     Logical Operators: 1
88     Bitwise Operators: 2
89     Assignment Operators: 7
90     Increment/Decrement Operators: 1
91 Delimiters: 19
92
93
94
95 Errors: 1
96
97 Error Details:
98 -----
99 Line 18: Unrecognized token -> @

```

Also , for robustness , I checked the analyzer with some intentional errorneus input files. The example are following:-

Listing 4: input2.txt

```

1
2 int a = 'abc';
3 print(a);
4 if (a @ b) {

```

```

5     a = a + 1;
6 }
7 string s = "hello

```

Listing 5: output2.txt

```

1 Token Stream:
2 -----
3
4 <KEYWORD , int>
5 <IDENTIFIER , a>
6 <ASSIGN_OP , =>
7 [ERROR] Line 1: Invalid character constant -> 'abc'
8 <DELIMITER , ;>
9 <KEYWORD , print>
10 <DELIMITER , (>
11 <IDENTIFIER , a>
12 <DELIMITER , )>
13 <DELIMITER , ;>
14 <KEYWORD , if>
15 <DELIMITER , (>
16 <IDENTIFIER , a>
17 [ERROR] Line 3: Unrecognized token -> @
18 <IDENTIFIER , b>
19 <DELIMITER , )>
20 <DELIMITER , {>
21 <IDENTIFIER , a>
22 <ASSIGN_OP , =>
23 <IDENTIFIER , a>
24 <ARITH_OP , +>
25 <INT_CONST , 1>
26 <DELIMITER , ;>
27 <DELIMITER , }>
28 <IDENTIFIER , string>
29 <IDENTIFIER , s>
30 <ASSIGN_OP , =>
31 [ERROR] Line 6: Unrecognized token -> "
32 <IDENTIFIER , hello>
33
34
35
36
37 Summary Section:
38 -----
39 Total Tokens: 26
40 Keywords: 3
41 Identifiers: 9

```

```
42 Constants: 1
43     Integer Constants: 1
44     Float Constants: 0
45     Character Constants: 0
46     String Constants: 0
47 Operators: 4
48     Arithmetic Operators: 1
49     Relational Operators: 0
50     Logical Operators: 0
51     Bitwise Operators: 0
52     Assignment Operators: 3
53     Increment/Decrement Operators: 0
54 Delimiters: 9
55
56
57
58 Errors: 3
59
60 Error Details:
61 -----
62 Line 1: Invalid character constant -> 'abc'
63 Line 3: Unrecognized token -> @
64 Line 6: Unrecognized token -> "
```